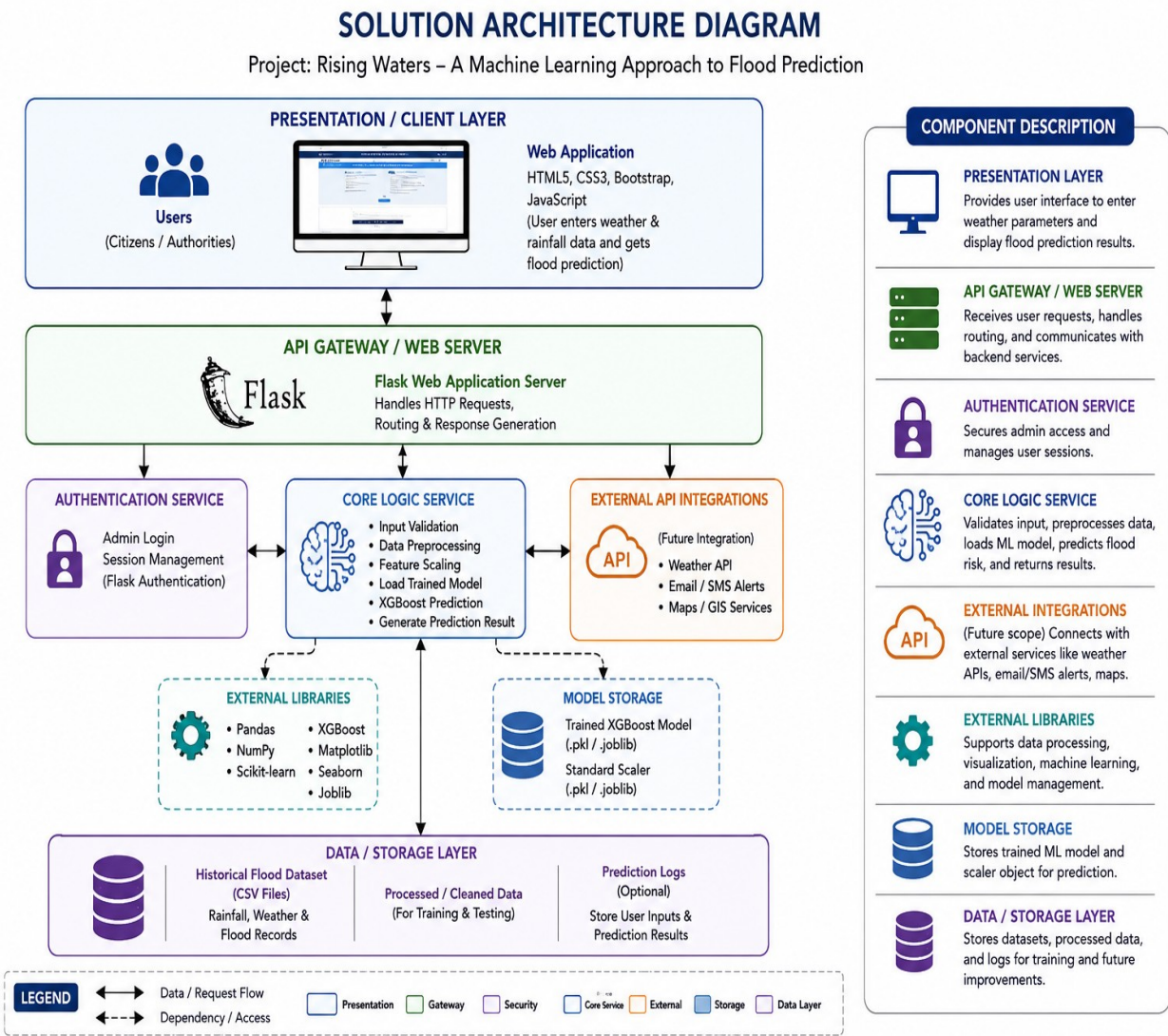


Project Design Phase

Solution Architecture

Date	17 July 2026
Team ID	-----
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	4 Marks

Solution Architecture Diagram



Architecture Flow

1. The **User** opens the Flask web application.
2. The user enters **rainfall and weather parameters**.
3. The **Flask server** receives the request.
4. The system validates the input values.
5. The **preprocessing module** performs feature scaling and formatting.
6. The trained **XGBoost model** is loaded from storage.
7. The model predicts whether there is a **Flood Risk** or **No Flood Risk**.
8. The prediction result is displayed to the user through the web interface.

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides a responsive web interface where users enter rainfall and weather parameters and view flood prediction results.	HTML5, CSS3, Bootstrap, JavaScript
API Gateway / Web Server	Receives user requests, routes them to the prediction service, validates inputs, and returns prediction results.	Flask (Python)
Authentication Service	Provides secure administrator login for model management and application maintenance. General users can access prediction without authentication.	Flask Session Authentication
Core Logic Service	Performs input validation, data preprocessing, feature scaling, loads the trained XGBoost model, predicts flood risk, and returns the result.	Python, XGBoost, Scikit-learn, Pandas, NumPy
External Libraries	Supports data analysis, visualization, preprocessing, and machine learning model execution.	Pandas, NumPy, Matplotlib, Seaborn, Joblib
Model Storage	Stores the trained XGBoost model and preprocessing scaler for deployment and prediction.	Joblib (.joblib), Pickle (.pkl)
Data Layer / Storage	Stores the historical flood dataset used for model training, testing, and prediction. Can also maintain prediction logs.	CSV Dataset, File System